PCD| Homework 1

Write a program that measures the time required to transfer various amounts of data (500 MB, 1 GB) using different transport protocols and different transfer methods. The program should include detailed measurements to analyze the performance of distributed networks using **TCP**, **UDP**, and **QUIC** protocols.

Implementation Requirements:

1. **Supported Protocols (as parameters for client and server):**
    - **TCP** (Transport Control Protocol):
        - Provides reliable connections with flow control and in-order packet delivery.
    - **UDP** (User Datagram Protocol):
        - Provides fast transport without flow control or error handling mechanisms.
    - **QUIC** (Quick UDP Internet Connections):
        - Provides fast, reliable, and secure connections with built-in multiplexing, avoiding head-of-line blocking issues.
        - You can use one of the following libraries: **ngtcp2**, **quiche**, **lsquic, msquic,** et.al.
2. **Message Size:**
    - Transfer data in blocks with sizes ranging from **1** to **65535 bytes**.
    - Test at least 5 different block sizes within this range for each protocol (TCP, UDP, QUIC). Example sizes: 1; 500; 1,000; 10,000; 65,535 bytes.
3. **Data Transfer Amounts:**
    - Transfer **500 MB** and **1 GB** of data.
    - You may use a large reusable buffer or simulate real-world data structures (e.g., "1 GB = 1.5 million WhatsApp messages, 4000 photos, or 10,000 emails").
4. **Hashing Function**
    - Both the client and the server must implement a hashing mechanism over the transferred data to validate the correctness and integrity of the implementation.
    - **For example:**
        - The client computes a SHA-256 hash of the entire data payload (e.g., the generated buffer or input file) before sending it, and prints the resulting hash.
        - The server reconstructs the received data (in the correct order, as applicable), computes a SHA-256 hash over the assembled payload, and prints the resulting hash.
    - **The hashes printed by the client and server must match for a successful transfer.**

5. **Transfer Mechanisms:**
    - **Streaming:**
        - Continuous transfer of data without waiting for confirmation after each message.
    - **Stop-and-Wait:**
        - Acknowledgment for each message before sending the next one.
        - Implementing **stop-and-wait** for **TCP** is optional since TCP already manages acknowledgments and retransmissions automatically.
        - Implementing **stop-and-wait** for **UDP** and **QUIC** is mandatory to analyze its impact on performance and data loss.

Output Requirements:

1. **The Server** must display the following at the end of each session:
    - The protocol used: **TCP**, **UDP**, or **QUIC**.
    - The total number of messages received.
    - The total number of bytes received.
2. **The Client** must display the following at the end of the execution:
    - Total transmission time (in seconds).
    - The total number of messages sent.
    - The total number of bytes sent.
3. **Testing and Report:**
    - Students must perform comparative tests using all three protocols (TCP, UDP, QUIC) for various message sizes and data amounts (500 MB and 1 GB).
    - The analysis must include detailed comparisons of:
        - Transmission time for each protocol.
        - Data loss rates (for UDP and QUIC).
        - The impact of message size on performance.

Evaluation Criteria:

1. Complete, functional client and server code that runs on a **Linux** system.
2. A PDF (Word/LaTeX) document containing:
    - Detailed information on how to use the program (options for client/server).
    - Statistics and comparisons of performance for each protocol.
    - A detailed analysis of the experimental results.
    - Conclusions about the advantages and disadvantages of each protocol under different transfer conditions.

General Observations:

1. If the buffer size for **UDP** is increased, data loss may be minimal in a **LAN**, but packet loss may occur in more complex networks.

2. The **stop-and-wait** mechanism is not very useful for **TCP**, as TCP already handles acknowledgments, but it may be interesting to analyze its impact.
3. The **stop-and-wait** mechanism is more relevant for **UDP** and **QUIC**, especially in contexts where message loss needs to be minimized.

📌 Instrucțiuni de trimitere a temei

📁 Format arhivă:

- **Nume fișier:** `Tema1_NumePrenume_2026_AcronimMaster.zip` *(fără diacritice)*
- **Exemplu:** `Tema1_PopescuIon_2026_MSD.zip`
- **Conținut:**
  - Cod sursă - propriu (fără librării)
  - Documentul PDF rezultat din Word sau LaTeX
- **Locație: URL-ul indicat în formularul de înregistrare (puteți modifica link-ul trimis doar dacă este absolut necesar).**